fatsd	main_freertos	MSP_EXP432E401Y_FREERTOS	ti_config_c	ti_config_h

I want this example project of fatsd example of msp432e401y to change this to configure fatfile sytem for internal flash 256kb last region

To redirect FatFs from the SPI SD Card to the last 256 KB of internal flash on the MSP432E401Y, you need to replace the physical disk I/O layer (`diskio.c` / `SDFatFS`) with an internal Flash block driver and protect that memory area in the linker script.

Step 1: Protect the Flash Region in the Linker Script

The MSP432E401Y has 1024 KB (1 MB) of on-chip Flash ranging from `0x00000000` to `0x00100000` .
The last 256 KB starts at `0x000C0000` (`0x00100000 - 0x00040000`).

MSP_EXP432E401Y_FREER... + 1

In `MSP_EXP432E401Y_FREERTOS.cmd` , reduce the application Flash allocation to 768 KB (`0x000C0000`) so the code and data do not overwrite the filesystem:

MSP_EXP432E401Y_FREER...

DOS

```
MEMORY
{
    /* Shrink application FLASH from 0x00100000 to 0x000C0000 (768 KB) */
    FLASH (RX) : origin = 0x00000000, length = 0x000C0000
    /* Reserve the last 256 KB for the FatFs storage volume */
    FLASH_FS (RW) : origin = 0x000C0000, length = 0x00040000
```

```
    SRAM (RWX) : origin = 0x20000000, length = 0x00040000
}
```

Step 2: Implement the Flash Disk I/O Driver

NOR Flash requires sector erasures (in MSP432E401Y, erase block size is **16 KB / 16384 bytes**) before writing. A standard FatFS sector is 512 bytes.

Create a new file `fatfs_flash_diskio.c` :

C

```c
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <third_party/fatfs/diskio.h>
#include <ti/devices/msp432e4/driverlib/flash.h>

#define FLASH_FS_BASE_ADDR     0x000C0000UL
#define FLASH_FS_TOTAL_SIZE    (256UL * 1024UL)   /* 256 KB */
#define FLASH_SECTOR_SIZE      512UL              /* FatFS logical sector */
#define FLASH_ERASE_BLOCK_SIZE 16384UL            /* Hardware erase block (16 KB)

#define FLASH_SECTOR_COUNT     (FLASH_FS_TOTAL_SIZE / FLASH_SECTOR_SIZE) /* 512 s

static uint8_t s_blockBuffer[FLASH_ERASE_BLOCK_SIZE];

DSTATUS disk_initialize(BYTE pdrv)
{
    if (pdrv != 0) return STA_NOINIT;
    return 0;
}

DSTATUS disk_status(BYTE pdrv)
{
    if (pdrv != 0) return STA_NOINIT;
    return 0;
}

DRESULT disk_read(BYTE pdrv, BYTE *buff, DWORD sector, UINT count)
{
```

```
    if (pdrv != 0 || (sector + count) > FLASH_SECTOR_COUNT) {
        return RES_PARERR;
    }

    uint32_t srcAddr = FLASH_FS_BASE_ADDR + (sector * FLASH_SECTOR_SIZE);
    memcpy(buff, (const void *)srcAddr, count * FLASH_SECTOR_SIZE);

    return RES_OK;
}

DRESULT disk_write(BYTE pdrv, const BYTE *buff, DWORD sector, UINT count)
{
    if (pdrv != 0 || (sector + count) > FLASH_SECTOR_COUNT) {
        return RES_PARERR;
    }

    for (UINT i = 0; i < count; i++) {
        uint32_t currentSector = sector + i;
        uint32_t sectorOffset = currentSector * FLASH_SECTOR_SIZE;
        uint32_t blockBaseOffset = (sectorOffset / FLASH_ERASE_BLOCK_SIZE) * FLAS
        uint32_t offsetInBlock = sectorOffset % FLASH_ERASE_BLOCK_SIZE;
        uint32_t blockPhysAddr = FLASH_FS_BASE_ADDR + blockBaseOffset;

        /* Buffer 16KB block, update 512B chunk, erase block, rewrite */
        memcpy(s_blockBuffer, (const void *)blockPhysAddr, FLASH_ERASE_BLOCK_SIZE
        memcpy(&s_blockBuffer[offsetInBlock], &buff[i * FLASH_SECTOR_SIZE], FLASH

        FlashErase(blockPhysAddr);
        FlashProgram((uint32_t *)s_blockBuffer, blockPhysAddr, FLASH_ERASE_BLOCK_
    }

    return RES_OK;
}

DRESULT disk_ioctl(BYTE pdrv, BYTE cmd, void *buff)
{
    if (pdrv != 0) return RES_PARERR;

    switch (cmd) {
        case CTRL_SYNC:
            return RES_OK;
        case GET_SECTOR_COUNT:
            *(DWORD *)buff = FLASH_SECTOR_COUNT;
            return RES_OK;
        case GET_SECTOR_SIZE:
            *(WORD *)buff = FLASH_SECTOR_SIZE;
```

```
            return RES_OK;
        case GET_BLOCK_SIZE:
            *(DWORD *)buff = FLASH_ERASE_BLOCK_SIZE / FLASH_SECTOR_SIZE; /* 32 se
            return RES_OK;
        default:
            return RES_PARERR;
    }
}
```

Step 3: Update `fatsd.c`

1. Remove `#include <ti/drivers/SDFatFS.h>` . fatsd.c
2. Include native FatFs headers `#include <third_party/fatfs/ff.h>` .
3. Replace `SDFatFS_open()` / `SDFatFS_close()` with standard `f_mount()` and auto-format via `f_mkfs()` if mounting fails (initial run). fatsd.c

Modify `mainThread` in `fatsd.c` :

c

```c
#include <third_party/fatfs/ff.h>
#include <third_party/fatfs/ffcio.h>
/* (Keep standard library, display, GPIO headers) */

static FATFS g_fatFs;

void *mainThread(void *arg0)
{
    FRESULT fresult;
    FILE *src, *dst;
    unsigned int bytesRead = 0, bytesWritten = 0, filesize = 0, totalBytesCopied

    GPIO_init();[cite: 1]
    Display_init();[cite: 1]
    /* SDFatFS_init(); is removed */

    GPIO_setConfig(CONFIG_GPIO_LED_0, GPIO_CFG_OUT_STD | GPIO_CFG_OUT_LOW);[cite:

    add_device(fatfsPrefix, _MSA, ffcio_open, ffcio_close, ffcio_read,
```

```
    ffcio_write, ffcio_lseek, ffcio_unlink, ffcio_rename);[cite: 1]

display = Display_open(Display_Type_UART, NULL);[cite: 1]
if (display == NULL) { while (1); }[cite: 1]

clock_settime(CLOCK_REALTIME, &ts);[cite: 1]
GPIO_write(CONFIG_GPIO_LED_0, CONFIG_GPIO_LED_ON);[cite: 1]

Display_printf(display, 0, 0, "Starting Internal Flash FatFS example\n");

/* Mount volume on drive 0 */
fresult = f_mount(&g_fatFs, "0:", 1);
if (fresult == FR_NO_FILESYSTEM) {
    Display_printf(display, 0, 0, "No filesystem found. Formatting 256KB flas
    BYTE workBuf[FF_MAX_SS];
    fresult = f_mkfs("0:", FM_FAT, 0, workBuf, sizeof(workBuf));
    if (fresult != FR_OK) {
        Display_printf(display, 0, 0, "Format failed (%d)!\n", fresult);
        while (1);
    }
    fresult = f_mount(&g_fatFs, "0:", 1);
}

if (fresult != FR_OK) {
    Display_printf(display, 0, 0, "Mount failed (%d)!\n", fresult);
    while (1);
}
Display_printf(display, 0, 0, "Internal flash drive 0 mounted successfully.\n

/* File operations using fopen/fread/fwrite/fclose remain identical */
src = fopen(inputfile, "r");[cite: 1]
if (!src) {
    Display_printf(display, 0, 0, "Creating a new file \"%s\"...", inputfile)
    src = fopen(inputfile, "w+");[cite: 1]
    if (!src) {
        Display_printf(display, 0, 0, "Error creating file\n");
        while (1);
    }
    fwrite(textarray, 1, strlen(textarray), src);[cite: 1]
    fflush(src);[cite: 1]
    rewind(src);[cite: 1]
    Display_printf(display, 0, 0, "done\n");[cite: 1]
}

dst = fopen(outputfile, "w");[cite: 1]
if (!dst) {
```

```
        Display_printf(display, 0, 0, "Error creating destination file\n");
        while (1);
    }

    while (true) {
        bytesRead = fread(cpy_buff, 1, CPY_BUFF_SIZE, src);[cite: 1]
        if (bytesRead == 0) break;[cite: 1]
        bytesWritten = fwrite(cpy_buff, 1, bytesRead, dst);[cite: 1]
        if (bytesWritten < bytesRead) {
            Display_printf(display, 0, 0, "Disk Full\n");[cite: 1]
            break;[cite: 1]
        }
        totalBytesCopied += bytesWritten;[cite: 1]
    }

    fflush(dst);[cite: 1]
    fseek(src, 0, SEEK_END);[cite: 1]
    filesize = ftell(src);[cite: 1]
    fclose(src);[cite: 1]
    fclose(dst);[cite: 1]

    Display_printf(display, 0, 0, "Copied %u bytes.\n", totalBytesCopied);

    /* Print back contents */
    dst = fopen(outputfile, "r");[cite: 1]
    while ((bytesRead = fread(cpy_buff, 1, CPY_BUFF_SIZE, dst)) > 0) {
        cpy_buff[bytesRead] = '\0';[cite: 1]
        Display_printf(display, 0, 0, "%s", cpy_buff);[cite: 1]
    }
    fclose(dst);[cite: 1]

    f_mount(NULL, "0:", 0);
    Display_printf(display, 0, 0, "Flash drive unmounted.\n");

    return NULL;
}
```

Step 4: SysConfig and FatFs Settings (`ffconf.h`)

1. Open your project's `.syscfg` file and **remove** the `SD (SDFatFS)` and `SPI` modules if they are not used elsewhere.
2. In `ffconf.h` , ensure:

`#define FF_USE_MKFS 1` (enables `f_mkfs()` to format the uninitialized flash).

`#define FF_FS_READONLY 0`

`#define FF_MIN_SS 512` and `#define FF_MAX_SS 512`

Why is LittleFS often preferred over FAT for 256KB internal flash, and how do I use it on MSP432E4?

Why LittleFS is Preferred Over FAT for On-Chip Flash

Feature	FAT (FatFs)	LittleFS	Why it Matters o MSP432E4 Flash
Power-Loss Resilience	Vulnerable. Metadata changes require multi-step overwrites. A reset during a write corrupts the File Allocation Table.	**Atomic & Crash-Safe.** Uses copy-on-write trees; sudden resets leave old files intact without corruption.	Crucial for emb systems withou shutdowns or b batteries.
Wear Leveling	**None.** FAT directory tables and cluster entries hit the exact same flash sectors repeatedly.	**Dynamic Wear Leveling.** Cycles writes across blocks and skips bad/worn areas automatically.	NOR flash block typically endure cycles. FAT can out block 0 in w logging.
Flash Erase Granularity	Expects 512-byte block overwrites. Requires emulating writes via read-erase-rewrite cycles.	Native awareness of **different read, program, and erase block sizes.**	MSP432E4 flash requires **16 KB b erases.** LittleFS handles write bu and block cyclir natively without custom cache l

Feature	FAT (FatFs)	LittleFS	Why it Matters o MSP432E4 Flash
RAM Footprint	Needs full sector and allocation buffers (~2 KB to 16 KB+ RAM).	Configurable, operates on tiny block caches (as low as 256–512 B RAM).	Frees up SRAM constrained tar

How to Use LittleFS on MSP432E4 (256 KB Flash Region)

1. Add LittleFS Source Files

Download the official LittleFS repository and add the core files to your CCS/IAR project:

`lfs.c` / `lfs.h`

`lfs_util.c` / `lfs_util.h`

2. Implement the Hardware Abstraction Layer (`lfs_port_msp432e4.c`)

The MSP432E401Y flash controller allows reading directly via memory pointers, programming in **4-byte (32-bit word)** increments, and erasing in **16 KB blocks** (`FlashErase()` / `FlashProgram()`).

Create `lfs_port_msp432e4.c` :

c

```c
#include <stdint.h>
#include <string.h>
#include "lfs.h"
#include <ti/devices/msp432e4/driverlib/flash.h>

#define LFS_FLASH_BASE_ADDR   0x000C0000UL      /* Start of last 256 KB */
#define LFS_BLOCK_SIZE        16384UL           /* 16 KB erase block size */
#define LFS_BLOCK_COUNT       16UL              /* 256 KB / 16 KB = 16 blocks */
#define LFS_PROG_SIZE         4UL               /* MSP432E4 flash word size (4 b
#define LFS_READ_SIZE         4UL               /* Read alignment */
#define LFS_CACHE_SIZE        64UL              /* Read/Prog cache buffer */
#define LFS_LOOKAHEAD_SIZE    16UL              /* 16 blocks -> 2 bytes rounded
```

```c
/* Static cache and lookahead buffers (no heap required) */
static uint8_t s_read_buf[LFS_CACHE_SIZE];
static uint8_t s_prog_buf[LFS_CACHE_SIZE];
static uint8_t s_lookahead_buf[LFS_LOOKAHEAD_SIZE];

/* Read callback */
static int msp432_flash_read(const struct lfs_config *c, lfs_block_t block,
                             lfs_off_t off, void *buffer, lfs_size_t size)
{
    uint32_t addr = LFS_FLASH_BASE_ADDR + (block * c->block_size) + off;
    memcpy(buffer, (const void *)addr, size);
    return LFS_ERR_OK;
}

/* Program callback */
static int msp432_flash_prog(const struct lfs_config *c, lfs_block_t block,
                             lfs_off_t off, const void *buffer, lfs_size_t size)
{
    uint32_t addr = LFS_FLASH_BASE_ADDR + (block * c->block_size) + off;

    /* FlashProgram requires 4-byte aligned count and addresses */
    int32_t status = FlashProgram((uint32_t *)buffer, addr, size);
    return (status == 0) ? LFS_ERR_OK : LFS_ERR_IO;
}

/* Erase callback */
static int msp432_flash_erase(const struct lfs_config *c, lfs_block_t block)
{
    uint32_t addr = LFS_FLASH_BASE_ADDR + (block * c->block_size);

    int32_t status = FlashErase(addr);
    return (status == 0) ? LFS_ERR_OK : LFS_ERR_IO;
}

/* Sync callback (flash writes are synchronous on MSP432E4) */
static int msp432_flash_sync(const struct lfs_config *c)
{
    return LFS_ERR_OK;
}

/* LittleFS configuration structure */
const struct lfs_config g_lfs_cfg = {
    .read  = msp432_flash_read,
    .prog  = msp432_flash_prog,
    .erase = msp432_flash_erase,
```

```
    .sync  = msp432_flash_sync,

    .read_size      = LFS_READ_SIZE,
    .prog_size      = LFS_PROG_SIZE,
    .block_size     = LFS_BLOCK_SIZE,
    .block_count    = LFS_BLOCK_COUNT,
    .cache_size     = LFS_CACHE_SIZE,
    .lookahead_size = LFS_LOOKAHEAD_SIZE,
    .block_cycles   = 500,

    .read_buffer      = s_read_buf,
    .prog_buffer      = s_prog_buf,
    .lookahead_buffer = s_lookahead_buf
};
```

3. Application Usage in `mainThread`

Replace the FatFs / stdio routines with the native LittleFS API:

C

```c
#include "lfs.h"
#include <ti/display/Display.h>

extern const struct lfs_config g_lfs_cfg;
static lfs_t lfs;
static lfs_file_t file;

void *mainThread(void *arg0)
{
    Display_Handle display = Display_open(Display_Type_UART, NULL);
    Display_printf(display, 0, 0, "Starting LittleFS on Internal Flash...\n");

    /* 1. Mount filesystem; format if first boot or corrupted */
    int err = lfs_mount(&lfs, &g_lfs_cfg);
    if (err) {
        Display_printf(display, 0, 0, "Formatting 256KB flash area...\n");
        lfs_format(&lfs, &g_lfs_cfg);
        lfs_mount(&lfs, &g_lfs_cfg);
    }
```

```
    /* 2. Open and update a boot counter */
    uint32_t boot_count = 0;
    lfs_file_open(&lfs, &file, "boot_count.bin", LFS_O_RDWR | LFS_O_CREAT);
    lfs_file_read(&lfs, &file, &boot_count, sizeof(boot_count));
    boot_count++;
    lfs_file_rewind(&lfs, &file);
    lfs_file_write(&lfs, &file, &boot_count, sizeof(boot_count));
    lfs_file_close(&lfs, &file);

    Display_printf(display, 0, 0, "Current Boot Count: %u\n", boot_count);

    /* 3. Write data to a text file */
    const char msg[] = "MSP432E4 LittleFS test successful!\n";
    lfs_file_open(&lfs, &file, "log.txt", LFS_O_WRONLY | LFS_O_CREAT | LFS_O_APPE
    lfs_file_write(&lfs, &file, msg, strlen(msg));
    lfs_file_close(&lfs, &file);

    /* 4. Read back and print the file */
    char read_buf[64];
    lfs_file_open(&lfs, &file, "log.txt", LFS_O_RDONLY);
    lfs_ssize_t read_bytes = lfs_file_read(&lfs, &file, read_buf, sizeof(read_buf
    if (read_bytes > 0) {
        read_buf[read_bytes] = '\0';
        Display_printf(display, 0, 0, "Read back:\n%s", read_buf);
    }
    lfs_file_close(&lfs, &file);

    lfs_unmount(&lfs);
    return NULL;
}
```

complete every changes what to be made clearly for fatfile system

To adapt the `fatsd` example on the MSP432E401Y to run the FAT filesystem entirely on the last 256 KB of internal flash, implement the modifications detailed below across your project files.

1. `MSP_EXP432E401Y_FREERTOS.cmd` (Linker Command File)

The total internal flash on the MSP432E401Y is 1024 KB (0x00000000 to 0x00100000). Shrink the executable flash region to 768 KB (`0x000C0000`) to prevent the code/data sections from overwriting the filesystem region. MSP_EXP432E401Y_FREER... + 2

Open `MSP_EXP432E401Y_FREERTOS.cmd` and modify the `MEMORY` block:

MSP_EXP432E401Y_FREER...

DOS

```
MEMORY
{
    /* Reduced from length = 0x00100000 to 0x000C0000 (768 KB) */
    FLASH (RX) : origin = 0x00000000, length = 0x000C0000

    /* Dedicated 256 KB area reserved for FatFs */
    FLASH_FS (RW) : origin = 0x000C0000, length = 0x00040000

    SRAM (RWX) : origin = 0x20000000, length = 0x00040000
}
```

2. SysConfig / Driver Setup (`fatsd.syscfg`)

In the Code Composer Studio Project:

1. Open the `.syscfg` file.
2. **Remove** the `SD (SDFatFS)` component. ti_config_c.txt + 1
3. **Remove** the `SPI` component assigned to the SD card (SSI2 / GPIO PC7) if not needed elsewhere. ti_config_c.txt + 1
4. Save the file. SysConfig will regenerate `ti_drivers_config.c` and `ti_drivers_config.h` without SD card dependencies. ti_config_c.txt + 1

3. `ffconf.h` (FatFs Configuration)

Locate `ffconf.h` in your project or SDK path and verify the following macros:

C

```
#define FF_USE_MKFS         1       /* Enables f_mkfs() to format uninitialized f
#define FF_FS_READONLY      0       /* Read/Write support */
#define FF_MIN_SS           512     /* Sector size */
```

```
#define FF_MAX_SS           512
#define FF_FS_TINY          0
#define FF_FS_RPATH         0
```

4. Create `fatfs_flash_diskio.c` (Hardware Flash Interface)

Add a new source file named `fatfs_flash_diskio.c` to your project. This connects FatFs low-level calls (`disk_initialize` , `disk_read` , `disk_write` , `disk_ioctl`) to the internal flash controller via MSP432E4 DriverLib.

c

```c
#include <stdint.h>
#include <stdbool.h>
#include <string.h>
#include <third_party/fatfs/diskio.h>
#include <ti/devices/msp432e4/driverlib/flash.h>

#define FLASH_FS_BASE_ADDR     0x000C0000UL        /* Base address of last 256KB
#define FLASH_FS_TOTAL_SIZE    (256UL * 1024UL)    /* 256 KB volume */
#define FLASH_SECTOR_SIZE      512UL               /* Standard FatFs sector size
#define FLASH_ERASE_BLOCK_SIZE 16384UL             /* MSP432E4 hardware erase blo
#define FLASH_SECTOR_COUNT     (FLASH_FS_TOTAL_SIZE / FLASH_SECTOR_SIZE) /* 512 s

/* 16 KB RAM bounce buffer for block read-modify-write operations */
static uint8_t s_blockBuffer[FLASH_ERASE_BLOCK_SIZE];

DSTATUS disk_initialize(BYTE pdrv)
{
    if (pdrv != 0) {
        return STA_NOINIT;
    }
    return 0;
}

DSTATUS disk_status(BYTE pdrv)
{
    if (pdrv != 0) {
        return STA_NOINIT;
    }
    return 0;
```

```
}

DRESULT disk_read(BYTE pdrv, BYTE *buff, DWORD sector, UINT count)
{
    if (pdrv != 0 || (sector + count) > FLASH_SECTOR_COUNT) {
        return RES_PARERR;
    }

    uint32_t srcAddr = FLASH_FS_BASE_ADDR + (sector * FLASH_SECTOR_SIZE);
    memcpy(buff, (const void *)srcAddr, count * FLASH_SECTOR_SIZE);

    return RES_OK;
}

DRESULT disk_write(BYTE pdrv, const BYTE *buff, DWORD sector, UINT count)
{
    if (pdrv != 0 || (sector + count) > FLASH_SECTOR_COUNT) {
        return RES_PARERR;
    }

    for (UINT i = 0; i < count; i++) {
        uint32_t currentSector = sector + i;
        uint32_t sectorByteOffset = currentSector * FLASH_SECTOR_SIZE;
        uint32_t blockBaseOffset = (sectorByteOffset / FLASH_ERASE_BLOCK_SIZE) *
        uint32_t offsetInBlock = sectorByteOffset % FLASH_ERASE_BLOCK_SIZE;
        uint32_t blockPhysAddr = FLASH_FS_BASE_ADDR + blockBaseOffset;

        /* Step A: Copy 16KB erase block into RAM */
        memcpy(s_blockBuffer, (const void *)blockPhysAddr, FLASH_ERASE_BLOCK_SIZE

        /* Step B: Modify the corresponding 512B sector */
        memcpy(&s_blockBuffer[offsetInBlock], &buff[i * FLASH_SECTOR_SIZE], FLASH

        /* Step C: Erase hardware block and rewrite updated content */
        FlashErase(blockPhysAddr);
        FlashProgram((uint32_t *)s_blockBuffer, blockPhysAddr, FLASH_ERASE_BLOCK_
    }

    return RES_OK;
}

DRESULT disk_ioctl(BYTE pdrv, BYTE cmd, void *buff)
{
    if (pdrv != 0) {
        return RES_PARERR;
    }
```

```
    switch (cmd) {
        case CTRL_SYNC:
            return RES_OK;
        case GET_SECTOR_COUNT:
            *(DWORD *)buff = FLASH_SECTOR_COUNT;
            return RES_OK;
        case GET_SECTOR_SIZE:
            *(WORD *)buff = FLASH_SECTOR_SIZE;
            return RES_OK;
        case GET_BLOCK_SIZE:
            *(DWORD *)buff = FLASH_ERASE_BLOCK_SIZE / FLASH_SECTOR_SIZE; /* 32 se
            return RES_OK;
        default:
            return RES_PARERR;
    }
}
```

5. `fatsd.c` (Application Code)

Replace the contents of `fatsd.c` with the following implementation. It removes all `SDFatFS` references, initializes the FatFs driver, formats on first boot via `f_mkfs()`, and runs the standard POSIX read/write test: fatsd.c

C

```c
#include <file.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

/* POSIX Header files */
#include <pthread.h>

/* FatFs core and CIO binding */
#include <third_party/fatfs/ff.h>
#include <third_party/fatfs/ffcio.h>
```

```c
#include <ti/display/Display.h>
#include <ti/drivers/GPIO.h>

/* Driver configuration */
#include "ti_drivers_config.h"

#ifndef CPY_BUFF_SIZE
#define CPY_BUFF_SIZE       2048
#endif

#define STR_(n)             #n
#define STR(n)              STR_(n)

#define DRIVE_NUM           0

const char inputfile[] = "fat:"STR(DRIVE_NUM)":input.txt";
const char outputfile[] = "fat:"STR(DRIVE_NUM)":output.txt";

const char textarray[] = \
"*********************************************************************\n"
"MSP432E401Y Internal Flash FAT Filesystem (256 KB Region)\n"
"Data read and written to on-chip flash memory successfully.\n"
"*********************************************************************\n";

static Display_Handle display;
static FATFS g_fatFs;

const struct timespec ts = {
    .tv_sec = 1469647026,
    .tv_nsec = 0
};

char fatfsPrefix[] = "fat";
unsigned char cpy_buff[CPY_BUFF_SIZE];

void *mainThread(void *arg0)
{
    FILE *src, *dst;
    unsigned int bytesRead = 0;
    unsigned int bytesWritten = 0;
    unsigned int filesize;
    unsigned int totalBytesCopied = 0;
    FRESULT fresult;

    GPIO_init();
```

```
    Display_init();

    GPIO_setConfig(CONFIG_GPIO_LED_0, GPIO_CFG_OUT_STD | GPIO_CFG_OUT_LOW);

    /* Bind TI RTS C-IO functions (fopen, fread, fwrite) to FatFs */
    add_device(fatfsPrefix, _MSA, ffcio_open, ffcio_close, ffcio_read,
        ffcio_write, ffcio_lseek, ffcio_unlink, ffcio_rename);

    display = Display_open(Display_Type_UART, NULL);
    if (display == NULL) {
        while (1);
    }

    clock_settime(CLOCK_REALTIME, &ts);
    GPIO_write(CONFIG_GPIO_LED_0, CONFIG_GPIO_LED_ON);

    Display_printf(display, 0, 0, "\n--- MSP432E4 Internal Flash FAT File System

    /* Mount the internal flash drive */
    fresult = f_mount(&g_fatFs, "0:", 1);
    if (fresult == FR_NO_FILESYSTEM) {
        Display_printf(display, 0, 0, "No filesystem found. Formatting 256KB flas
        BYTE workBuf[FF_MAX_SS];
        fresult = f_mkfs("0:", FM_FAT, 0, workBuf, sizeof(workBuf));
        if (fresult != FR_OK) {
            Display_printf(display, 0, 0, "f_mkfs failed with code: %d\n", fresul
            while (1);
        }
        Display_printf(display, 0, 0, "Format complete. Re-mounting...\n");
        fresult = f_mount(&g_fatFs, "0:", 1);
    }

    if (fresult != FR_OK) {
        Display_printf(display, 0, 0, "Mount failed with code: %d\n", fresult);
        while (1);
    }

    Display_printf(display, 0, 0, "Drive %u mounted successfully.\n", DRIVE_NUM);

    /* Open or generate input file */
    src = fopen(inputfile, "r");
    if (!src) {
        Display_printf(display, 0, 0, "Creating \"%s\"...\n", inputfile);
        src = fopen(inputfile, "w+");
        if (!src) {
            Display_printf(display, 0, 0, "Error: could not create \"%s\"\n", inp
```

```
            while (1);
        }
        fwrite(textarray, 1, strlen(textarray), src);
        fflush(src);
        rewind(src);
        Display_printf(display, 0, 0, "File created.\n");
    } else {
        Display_printf(display, 0, 0, "Using existing copy of \"%s\"\n", inputfil
    }

    /* Copy inputfile to outputfile */
    dst = fopen(outputfile, "w");
    if (!dst) {
        Display_printf(display, 0, 0, "Error opening \"%s\"\n", outputfile);
        while (1);
    }

    while (true) {
        bytesRead = fread(cpy_buff, 1, CPY_BUFF_SIZE, src);
        if (bytesRead == 0) {
            break;
        }

        bytesWritten = fwrite(cpy_buff, 1, bytesRead, dst);
        if (bytesWritten < bytesRead) {
            Display_printf(display, 0, 0, "Disk Full\n");
            break;
        }
        totalBytesCopied += bytesWritten;
    }

    fflush(dst);

    fseek(src, 0, SEEK_END);
    filesize = ftell(src);
    rewind(src);

    fclose(src);
    fclose(dst);

    Display_printf(display, 0, 0, "Copied %u bytes to \"%s\".\n", totalBytesCopie

    /* Read back content from destination file to UART */
    dst = fopen(outputfile, "r");
    if (!dst) {
        Display_printf(display, 0, 0, "Error re-opening \"%s\"\n", outputfile);
```

```
            while (1);
        }

        while (true) {
            bytesRead = fread(cpy_buff, 1, CPY_BUFF_SIZE, dst);
            if (bytesRead == 0) {
                break;
            }
            cpy_buff[bytesRead] = '\0';
            Display_printf(display, 0, 0, "%s", cpy_buff);
        }

        fclose(dst);

        f_mount(NULL, "0:", 0);
        Display_printf(display, 0, 0, "Drive %u unmounted successfully.\n", DRIVE_NUM

        return NULL;
    }

    int32_t fatfs_getFatTime(void)
    {
        time_t seconds = time(NULL);
        struct tm *pTime = localtime(&seconds);

        uint32_t fatTime = ((uint32_t)(pTime->tm_year - 80) << 25) |
                           ((uint32_t)(pTime->tm_mon) << 21) |
                           ((uint32_t)(pTime->tm_mday) << 16) |
                           ((uint32_t)(pTime->tm_hour) << 11) |
                           ((uint32_t)(pTime->tm_min) << 5) |
                           ((uint32_t)(pTime->tm_sec) >> 1);

        return ((int32_t)fatTime);
    }
```

ffconf.h where this file is located

`ffconf.h` is located inside your **SimpleLink MSP432E4 SDK** installation directory rather than your local project folder by default.

Depending on where the SDK was installed, you will find it at:

Windows:

```
C:\ti\simplelink_msp432e4_sdk_<version>\source\third_party\fatfs\ffconf.h
```

Linux / macOS:

```
/home/<user>/ti/simplelink_msp432e4_sdk_<version>/source/third_party/fatfs/ffconf.h
```

Best Practice for Modifying It

Instead of editing the SDK's master file directly (which modifies it for all projects and requires rebuilding the SDK libraries):

1. **Copy** `ffconf.h` from the path above directly into your Code Composer Studio project's root directory.
2. In CCS, open **Project Properties → Build → ARM Compiler → Include Options**.
3. Ensure the project root `${PROJECT_ROOT}` is listed **before/above** the SDK include paths in the search order. This ensures the compiler picks up your local project copy of `ffconf.h` with `FF_USE_MKFS` enabled.

error #10056: symbol "disk_status" redefined: first defined in "./fatfs_flash_diskio.obj"; redefined in "C:/ti/simplelink_msp432e4_sdk_4_20_00_12/source/third_party/fatfs/lib/ccs/m4f/fatfs.a<diskio.obj>"

This error happens because the pre-compiled library `fatfs.a` in the SDK already contains TI's default `diskio.obj` implementation (which acts as a dispatcher pointing to `SD.h` / `SDFatFS`). When you added `disk_read`, `disk_write`, etc., inside `fatfs_flash_diskio.c`, the linker found duplicate definitions.

In TI's FatFs port, `diskio.c` delegates requests to media-specific function tables via `diskio_init()` or uses weak wrappers depending on the configuration.

You can resolve this using either of the two standard solutions below. **Option 1 is the cleanest and fastest.**

Option 1: Use the TI Linker `--reread_libs` / Priority Mechanism (No SDK modifications)

Tell the TI linker to accept your local `fatfs_flash_diskio.obj` as the primary definition and ignore the one inside `fatfs.a`.

1. In CCS, right-click your project → **Properties**.
2. Navigate to **ARM Linker → Advanced Options → Linker Output**.
3. Check the box for **"Allow multiple definitions"** (or in Command Line flags add `--relaxed_ref_elim=on` and `--warn_sections`).

Alternatively (and more reliably):

Rename the functions in `fatfs_flash_diskio.c` to hook into TI's internal dispatch table: TI's `diskio.obj` inside `fatfs.a` uses a table of function pointers or calls `disk_read`, etc. To bypass `diskio.obj` completely, instruct the linker to exclude `diskio.obj` from the archive:

1. Right-click project → **Properties → ARM Linker → File Search Path**.
2. Under **"Reread libraries to resolve reference" (** `--reread_libs` **)**, make sure it is checked.
3. In your project, rename `fatfs_flash_diskio.c` to `diskio.c`. Because CCS compiles local source files to object files (`diskio.obj`) in the `Debug` folder, the linker will see the local `diskio.obj` first. However, to avoid symbol clash with an archive object having the same symbols, use **Option 2**.

Option 2: Exclude `fatfs.a` and Compile FatFs Sources Directly in Your Project (Recommended)

Since you modified `ffconf.h` to enable `FF_USE_MKFS`, the precompiled library `fatfs.a` was compiled with the SDK's original `ffconf.h` anyway (where `FF_USE_MKFS` was disabled). Using the precompiled `.a` library will cause `f_mkfs` to be omitted or cause mismatch bugs.

Compile the FatFs source files directly in your project:

1. Exclude the pre-compiled `fatfs.a` library from the linker

In CCS, go to **Project Properties → ARM Linker → File Search Path**.

In the **"Include library file or command file as input"** list, find `fatfs.a` (or `${COM_TI_SIMPLELINK_MSP432E4_SDK_INSTALL_DIR}/source/third_party/fatfs/lib/...`) and **delete/remove** it.

2. Add the native FatFs C files into your project

Copy (or drag-and-drop as "Link to files") the following files from `C:/ti/simplelink_msp432e4_sdk_4_20_00_12/source/third_party/fatfs/` into your CCS project:

- `ff.c`
- `ffcio.c`
- `ffunicode.c` (if Unicode is enabled, otherwise optional)

Do NOT add `diskio.c` **from that SDK folder**, because your `fatfs_flash_diskio.c` is your custom `diskio.c` !

3. Rebuild

Clean the project: **Project → Clean...**

Build the project.

Now, `ff.c` will bind directly to your `fatfs_flash_diskio.c` disk functions, pick up your local `ffconf.h` , and link without duplicate symbol errors.